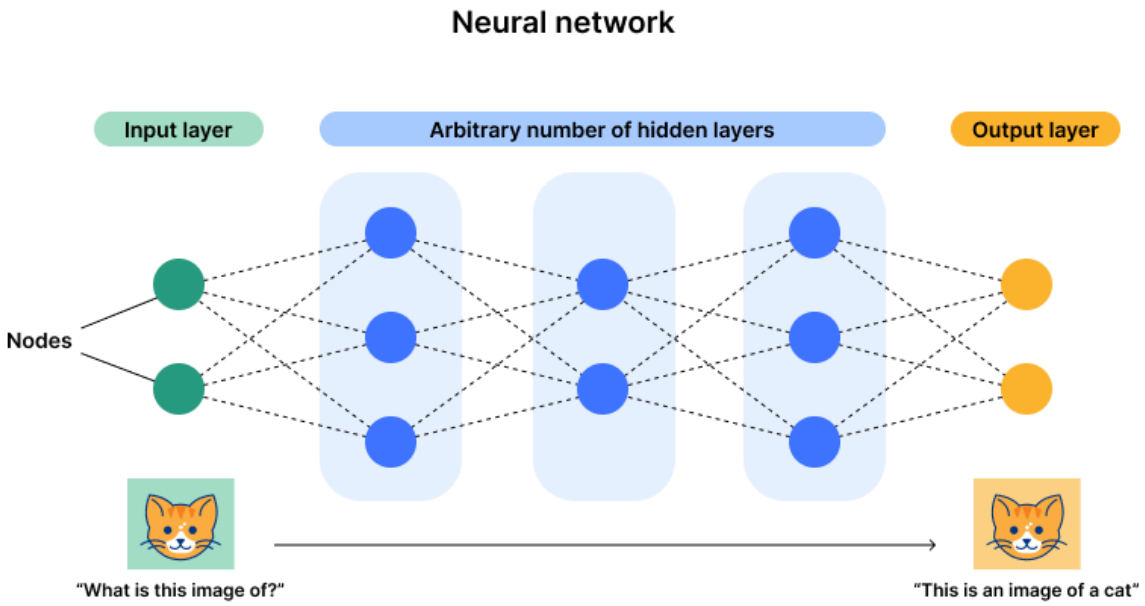
Unit- III

Neural Networks: The perceptron- learning rate, bias, perceptron learning algorithm, linear separability-Exclusive-OR function, backpropagation algorithm, initialization of weights, MLP in practice, deriving back propagation, radial basis functions. An illustrative example: face recognition.

Neural Network

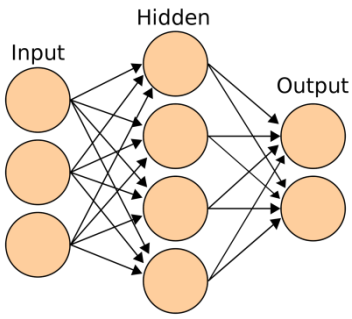
A neural network, or artificial neural network, is a type of computing architecture that is based on a model of how a human brain functions — hence the name "neural." Neural networks are made up of a collection of **processing units called "nodes**." These nodes pass data to each other, just like how in a brain, neurons pass electrical impulses to each other.



How do neural networks work:

Neural networks are composed of a collection of nodes. The nodes are spread out across at least three layers. The three layers are:

- An input layer
- A "hidden" layer
- An output layer



These three layers are the minimum. Neural networks can have more than one hidden layer, in addition to the input layer and output layer.

No matter which layer it is part of, each node performs some sort of processing task or function on whatever input it receives from the previous node (or from the input layer). Essentially, each node contains a mathematical formula, with each variable within the formula weighted differently. If the output of applying that mathematical formula to the input exceeds a certain threshold, the node passes data to the next layer in the neural network. If the output is below the threshold, no data is passed to the next layer.

Weights

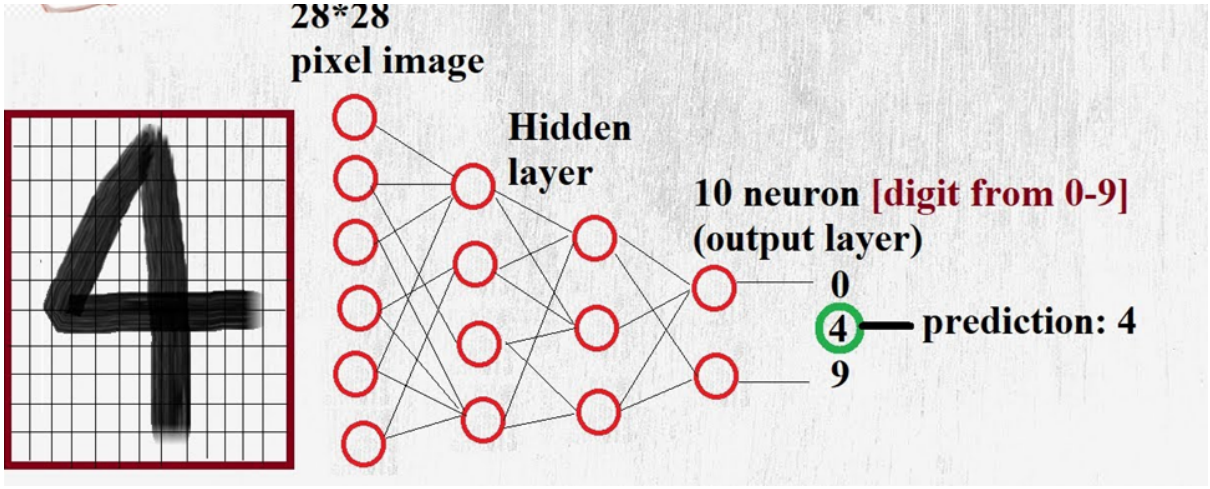
Weights are numerical values associated with the connections between neurons. They determine the strength of these connections and, in turn, the influence that one neuron's output has on another neuron's input. **Think of weights as the coefficients that adjust the impact of incoming data.** They can increase or decrease the importance of specific information.

During the training phase of a neural network, these weights are adjusted iteratively to minimize the difference between the network's predictions and the actual outcomes.

Let's consider a practical example to illustrate the role of weights. Suppose you're building a neural network to recognize handwritten digits. Each pixel in an image of a digit can be considered an input to the network. The weights associated with each pixel determine how much importance the network places on that pixel when making a decision about which digit is represented in the image.

As the network learns from a dataset of labeled digits, it adjusts these weights to give more significance to pixels that are highly correlated with the correct digit and less significance to pixels that are less relevant. Over time, the network learns to recognize patterns in the data and make accurate predictions.

In essence, weights are the neural network's way of learning from data. They capture the relationships between input features and the target output, allowing the network to generalize and make predictions on new, unseen data.



II. Bias

While weights determine the **strength of connections between neurons**, biases provide a critical **additional layer of flexibility** to neural networks. **Biases are essentially constants associated with each neuron**. Unlike weights, biases are not connected to specific inputs but are added to the neuron's output.

Biases serve as a form of offset or threshold, allowing neurons to activate even when the weighted sum of their inputs is not sufficient on its own. They introduce a level of adaptability that ensures the network can learn and make predictions effectively.

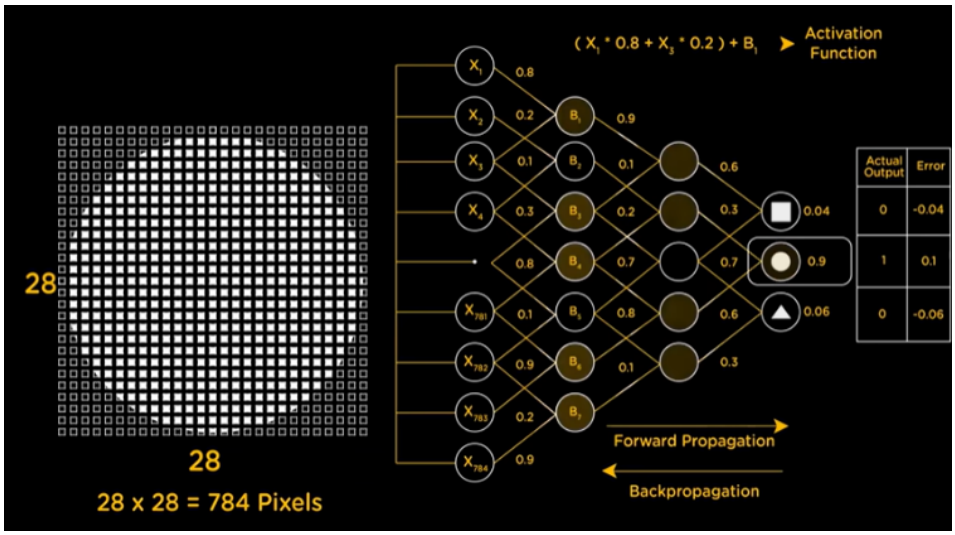
To understand the role of biases, consider a simple example. Imagine a neuron that processes the brightness of an image pixel. Without a bias, this neuron might only activate when the pixel's brightness is exactly at a certain threshold. However, by introducing a bias, you allow the neuron to activate even when the brightness is slightly below or above the threshold.

This flexibility is crucial because real-world data is rarely perfectly aligned with specific thresholds. Biases enable neurons to activate in response to various input conditions, making neural networks more robust and capable of handling complex patterns.

During training, biases are also adjusted to optimize the network's performance. They can be thought of as fine-tuning parameters that help the network fit the data better.

III. The Learning Process: Forward and Backward Propagation

Now that we understand the roles of weights and biases, let's explore how they come into play during the learning process of a neural network.



A. Forward Propagation

Forward propagation is the initial phase of processing input data through the neural network to produce an output or prediction. Here's how it works:

1. **Input Layer:** The input data is fed into the neural network's input layer.

1. **Weighted Sum:** Each neuron in the subsequent layers calculates a weighted sum of the inputs it receives, where the weights are the adjustable parameters.

1. **Adding Biases:** To this weighted sum, the bias associated with each neuron is added. This introduces an offset or threshold for activation.

1. **Activation Function:** The result of the weighted sum plus bias is passed through an activation function. **This function determines whether the neuron should activate or remain inactive based on the calculated value.**

1. **Propagation:** The output of one layer becomes the input for the next layer, and the process repeats until the final layer produces the network's prediction.

B. Backward Propagation

Once the network has made a prediction, it's essential to evaluate how accurate that prediction is and make adjustments to improve future predictions. This is where backward propagation comes into play:

1. **Error Calculation:** The prediction made by the network is compared to the actual target or ground truth. The resulting error, often quantified as a loss or cost, measures the disparity between prediction and reality.
2. **Gradient Descent:** During training, weights and biases are adjusted through an optimization process, often using a technique called gradient descent. The network calculates the gradient

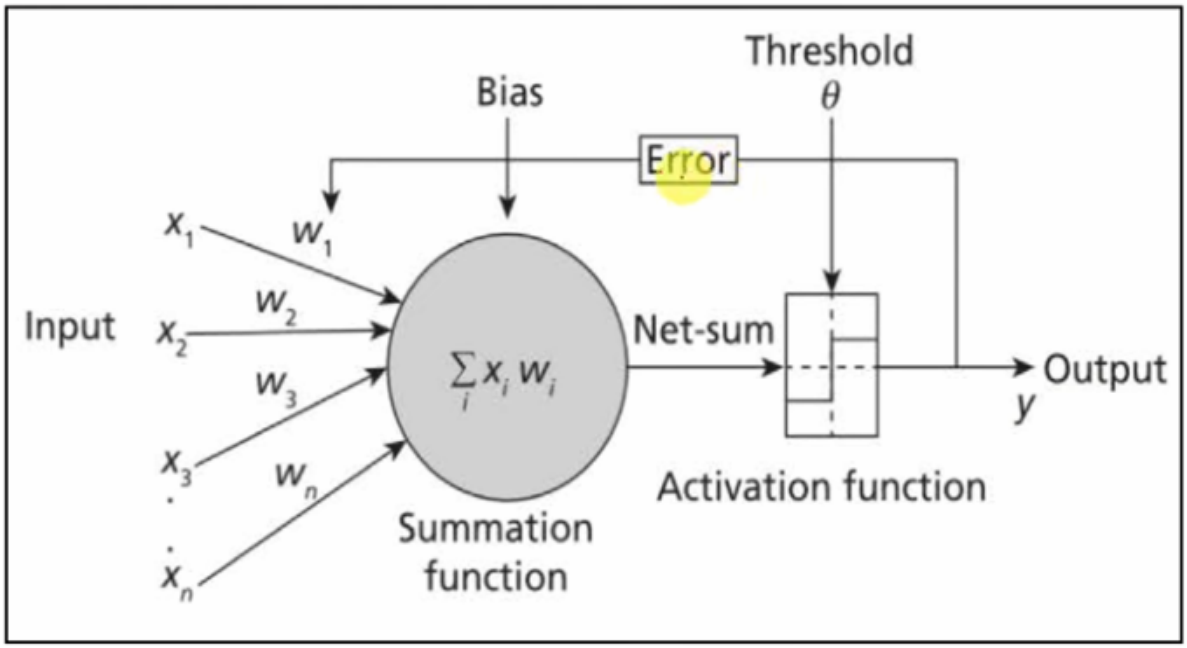
of the error (the difference between its predictions and the true values) with respect to the weights and biases. This gradient points in the direction of the steepest decrease in error.

1. **Bias Updates:** The network uses this gradient information to update the weights and biases throughout the network. The goal is to find the values that minimize the error.

1. **Iterative Process:** This process of forward and backward propagation is repeated iteratively on batches of training data. With each iteration, the network's weights and biases get closer to values that minimize the error.

The Perceptron

- A perceptron is the simplest form of an artificial neuron
- A perceptron takes multiple inputs, multiplies each by a weight, adds a bias, and passes the result through an activation function to produce an output.



How Does a Perceptron Work?

The perceptron works by receiving inputs, which could be feature values from a dataset, and combining them with a set of weights. These weights represent the strength of the connection between the inputs and the neuron. The perceptron's formula can be expressed as:

output = f(w1*x1 + w2*x2 + ... + wn*xn + b)

where:

- **w1, w2, ..., wn** are the weights,
- **x1, x2, ..., xn** are the input signals,
- **b** is the bias, which allows the activation function to be shifted to the left or right, to better fit the data,
- **f** is the activation function, typically a step function that outputs either 0 or 1.

The perceptron's decision-making process is binary. If the sum of the weighted inputs plus the bias is greater than zero, the perceptron outputs a 1; otherwise, it outputs a 0. This binary step function is what allows the perceptron to classify input data.

- A perceptron can only solve linearly separable problems like AND, OR, and NOT gates, but fails on non-linear problems like XOR.

Perceptron Example

Imagine a perceptron (in your brain).

The perceptron tries to decide if you should go to a concert.

Is the artist good? Is the weather good?

What weights should these facts have?

Criteria	Input	Weight
Artists is Good	**x1** = 0 or 1	**w1** = 0.7
Weather is Good	**x2** = 0 or 1	**w2** = 0.6
Friend will Come	**x3** = 0 or 1	**w3** = 0.5
Food is Served	**x4** = 0 or 1	**w4** = 0.3

.**The Perceptron Algorithm**

Frank Rosenblatt suggested this algorithm:

1. Set a threshold value:

- Threshold = 1

2. Multiply all inputs with its weights:

- x1 * w1 = 1 * 0.7 = 0.7
- x2 * w2 = 0 * 0.6 = 0
- x3 * w3 = 1 * 0.5 = 0.5
- x4 * w4 = 0 * 0.3 = 0

3. Sum all the results:

- 0.7 + 0 + 0.5 + 0 = 1.2 (The Weighted Sum)

4. Activate the Output:

- Return true if the sum > 1 ("Yes I will go to the Concert")

Training a Perceptron

Training a perceptron involves adjusting the weights and the bias based on the perceptron's performance on training data. **The perceptron learning rule**, also known as **the delta rule**, updates the weights and bias according to the following rule:

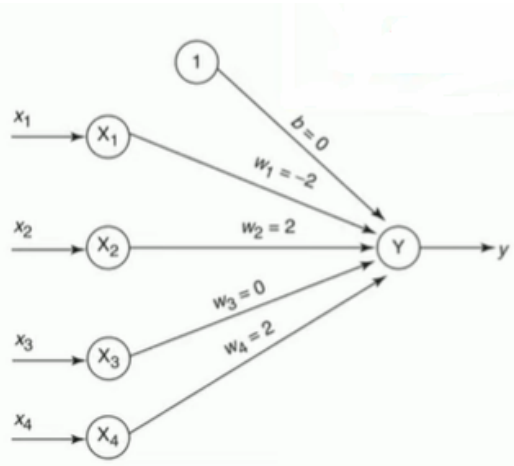
wi(new) = wi(old) + learning_rate * (expected_output - predicted_output) * xi

b(new) = b(old) + learning_rate * (expected_output - predicted_output)

where:

- **wi(new)** and **wi(old)** are the new and old weights, respectively,
- **learning_rate** is a small positive value that controls the magnitude of the weight update,
- **expected_output** is the true class label from the training data,
- **predicted_output** is the output produced by the perceptron,
- **xi** is the current input value.

The learning rate is a hyperparameter that determines how quickly the perceptron can adjust its weights. A small learning rate may lead to slow convergence, whereas a large learning rate may cause the perceptron to overshoot the optimal solution.



Input					Target
x_1	x_2	x_3	x_4	b	(t)
1	1	1	1	1	1
−1	1	−1	−1	1	1
1	1	1	−1	1	−1
1	−1	−1	1	1	−1

- Find the weights required to perform the following classification using perceptron network.
- The vectors (1, 1, 1, 1) and (- 1, 1 - 1, - 1) are belonging to the class 1, vectors (1, 1, 1, - 1) and (1, - 1, - 1, 1) are belonging to the class -1.
- Assume learning rate as 1
- and Initial weights as 0.

Input					Target
x_1	x_2	x_3	x_4	b	(t)
1	1	1	1	1	1
−1	1	−1	−1	1	1
1	1	1	−1	1	−1
1	−1	−1	1	1	−1

$$y_{in} = b + x_1 w_1 + x_2 w_2 + x_3 w_3 + x_4 w_4$$

$$y = f(y_{in}) = \begin{cases} 1 & \text{if } y_{in} > 0 \\ 0 & \text{if } y_{in} = 0 \\ -1 & \text{if } y_{in} < 0 \end{cases}$$

$$\Delta w_1 = \alpha t x_1;$$
$$\Delta w_2 = \alpha t x_2;$$
$$\Delta w_3 = \alpha t x_3;$$
$$\Delta w_4 = \alpha t x_4;$$
$$\Delta b = \alpha t$$

$$y_{in} = b + x_1 w_1 + x_2 w_2 + x_3 w_3 + x_4 w_4$$

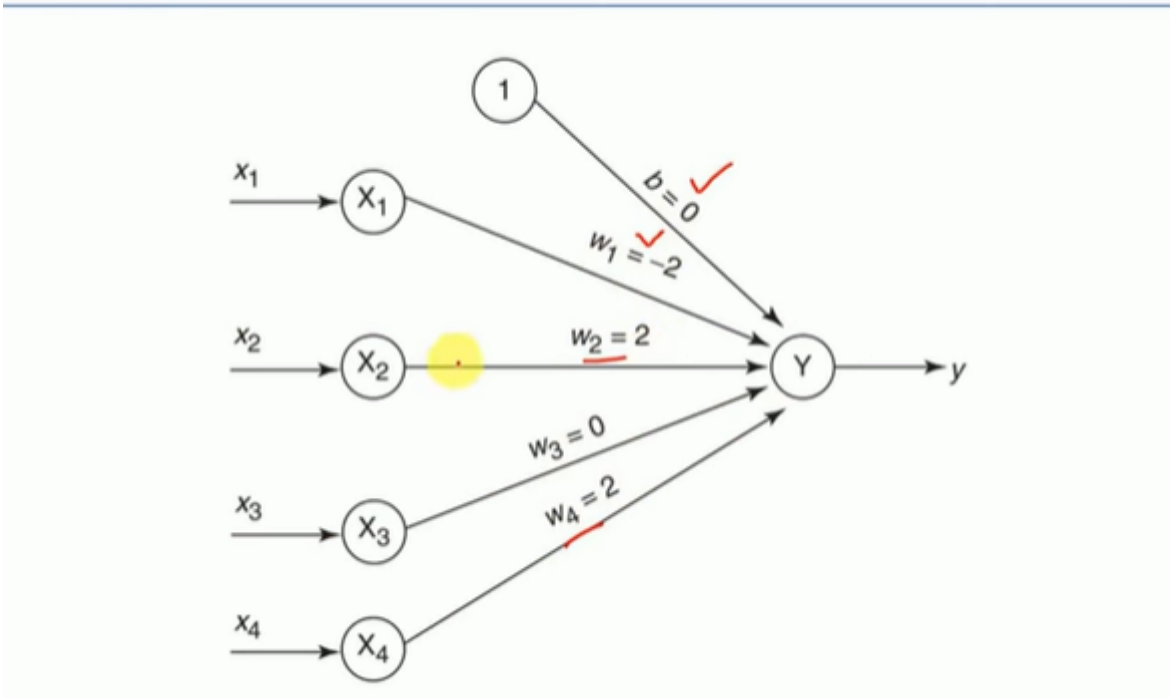
$$y = f(y_{in}) = \begin{cases} 1 & \text{if } y_{in} > 0 \\ 0 & \text{if } y_{in} = 0 \\ -1 & \text{if } y_{in} < 0 \end{cases}$$

Inputs				Target	Net input	output	Weight changes					Weights				
(x_1	x_2	x_3	x_4	(t)	(y_{in})	(y)	(Δw_1	Δw_2	Δw_3	Δw_4	Δb)	w_1 (0	w_2 0	w_3 0	w_4 0	b 0)
EPOCH-1																
(1	1	1	1	1	0	0	1	1	1	1	1	1	1	1	1	1
(-1	1	-1	-1	1	-1	-1	-1	1	-1	-1	1	0	2	0	0	2
(1	1	1	-1	-1	4	1	-1	-1	-1	1	-1	-1	1	-1	1	1
(1	-1	-1	1	-1	1	1	-1	1	1	-1	-1	-2	2	0	0	0

$$\Delta w_1 = \alpha t x_1;$$
$$\Delta w_2 = \alpha t x_2;$$
$$\Delta w_3 = \alpha t x_3;$$
$$\Delta w_4 = \alpha t x_4;$$
$$\Delta b = \alpha t$$

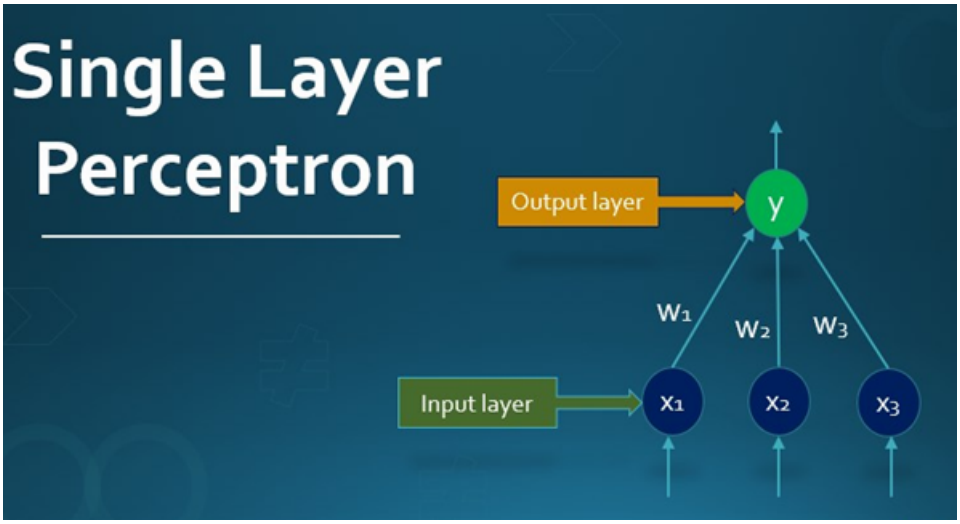
EPOCH-2																
(1	1	1	1	1	0	0	1	1	1	1	1	-1	3	1	1	1
(-1	1	-1	-1	1	3	1	0	0	0	0	0	-1	3	1	1	1
(1	1	1	-1	-1	4	1	-1	-1	-1	1	-1	-2	2	0	2	0
(1	-1	-1	1	-1	-2	-1	0	0	0	0	0	-2	2	0	2	0

EPOCH-3																
(1	1	1	1	1	2	1	0	0	0	0	0	-2	2	0	2	0
(-1	1	-1	-1	1	2	1	0	0	0	0	0	-2	2	0	2	0
(1	1	1	-1	-1	-2	-1	0	0	0	0	0	-2	2	0	2	0
(1	-1	-1	1	-1	-2	-1	0	0	0	0	0	-2	2	0	2	0



Types of perceptron

Single-Layer Perceptron is a type of perceptron is limited to learning linearly separable patterns. It is effective for tasks where the data can be divided into distinct categories through a straight line. While powerful in its simplicity, it struggles with more complex problems where the relationship between inputs and outputs is non-linear.

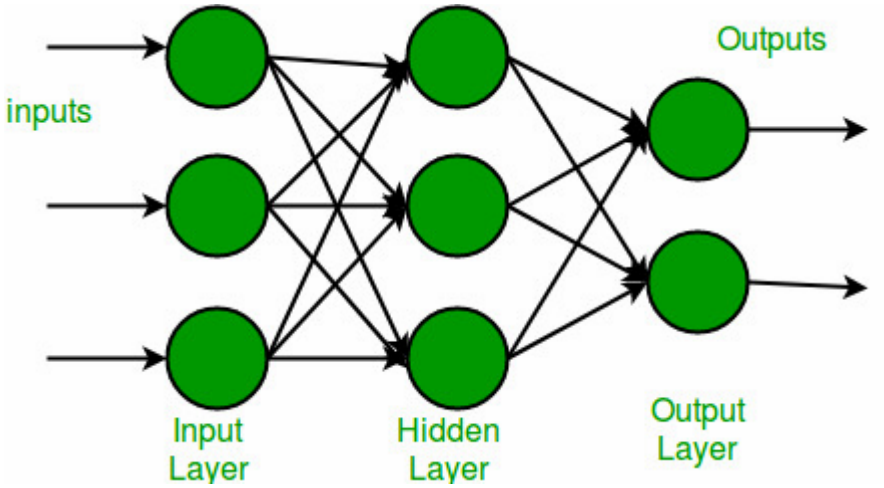


Multi-Layer Perceptron (MLP)

Multi-Layer Perceptron possess enhanced processing capabilities as they consist of two or more layers, adept at handling more complex patterns and relationships within the data.

- **Input Layer**: Each neuron or node in this layer corresponds to an input feature. For instance, if you have three input features the input layer will have three neurons.
- **Hidden Layers**: MLP can have any number of hidden layers with each layer containing any number of nodes. These layers process the information received from the input layer.

- **Output Layer**: The output layer generates the final prediction or result. If there are multiple outputs, the output layer will have a corresponding number of neurons.

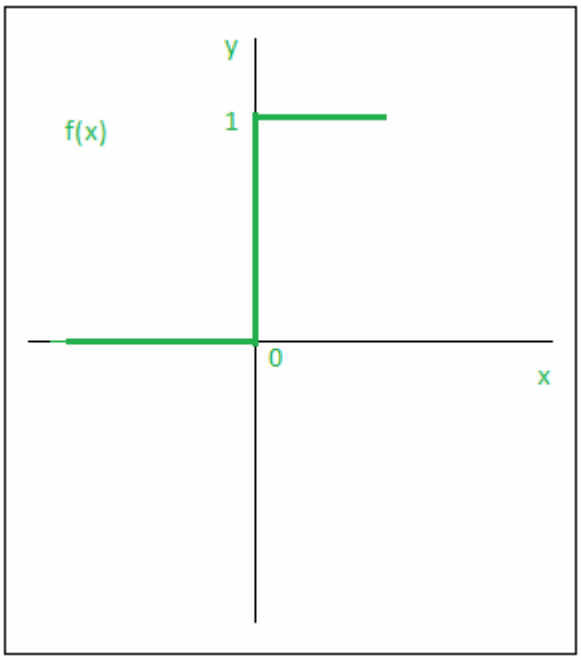


Every connection in the diagram is a representation of the fully connected nature of an MLP. This means that every node in one layer connects to every node in the next layer. As the data moves through the network each layer transforms it until the final output is generated in the output layer.

Types of Activation Functions –
Several different types of activation functions are used in Deep Learning. Some of them are explained below:

Step Function: Step Function is one of the simplest kind of activation functions. In this, we consider a threshold value and if the value of net input say **y** is greater than the threshold then the neuron is activated. Mathematically,

f(x)=1,if x>=0*f*(*x*)=1,if x>=0
f(x)=0,if x<0*f*(*x*)=0,if x<0

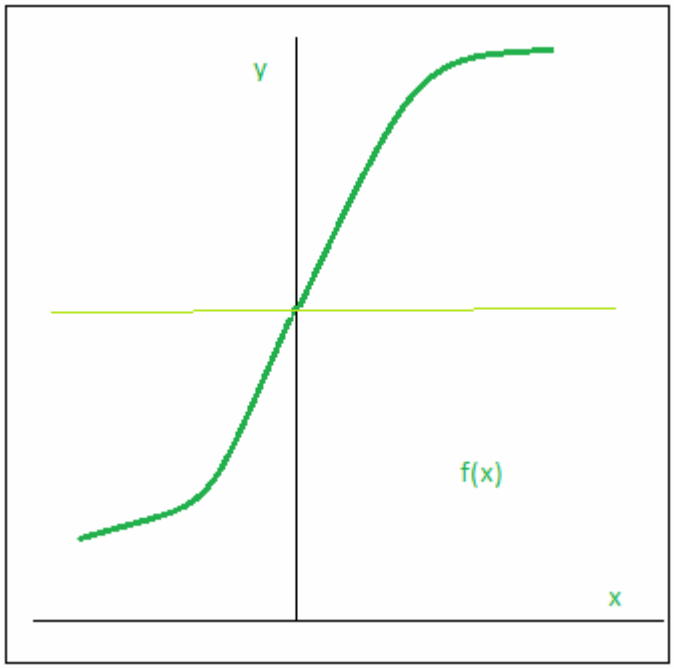


Sigmoid Function: Sigmoid function is a widely used activation function. It is defined as:

Formula: sigma(x) = 1 / (1 + exp(-x))

Where **x** is the weighted sum

Graphically,



It takes any input number and converts it into a value between 0 and 1.

For example:

- Input = **-10** → Output ≈ **0**
- Input = **0** → Output = **0.5**
- Input = **10** → Output ≈ **1**

Example: Pass or Fail

Suppose we want a neural network to predict whether a student will **PASS or FAIL**.
The neural network calculates some value:

Sigmoid converts it:

So:
0.88 = 88% chance of PASS
We can use a threshold of **0.5**:

Sigmoid Output	**Prediction**
< 0.5	FAIL
≥ 0.5	PASS

Therefore:

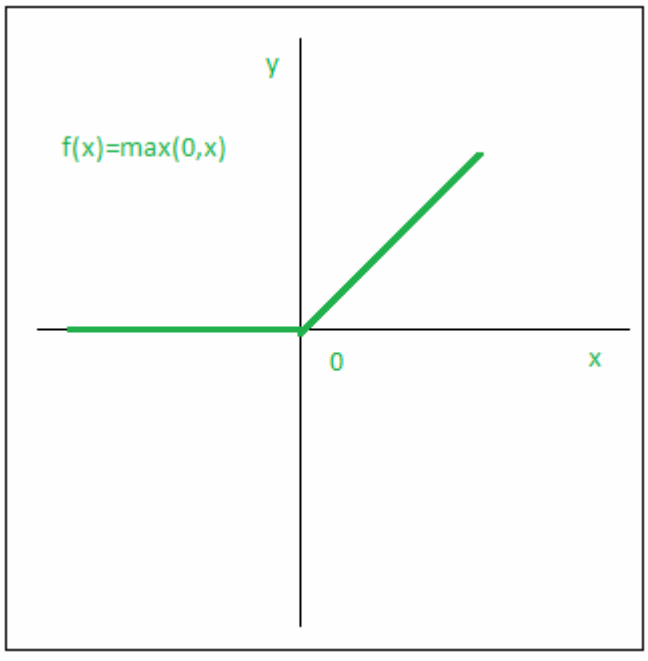
- **Prediction = PASS**

Note: Negative→0, Zero→0.5, positive →1

ReLU: The ReLU function is the Rectified linear unit. It is the most widely used activation function. It is defined as:

$$f(x)=\max(0,x)$$

Graphically,



The main advantage of using the ReLU function over other activation functions is that it does not activate all the neurons at the same time.

If you look at the ReLU function if the input is negative it will convert it to zero and the neuron does not get activated.
Computational time is less.

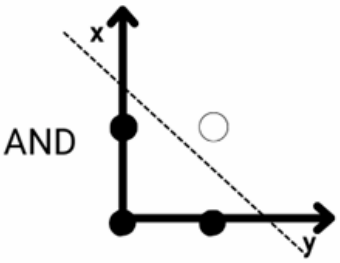
Linear Separability

Perceptron Rule For Exclusive-OR Function

Logic gates are the basic building blocks of digital circuits. They decide which set of input signals will trigger the circuit using boolean operations. They commonly have two inputs in binary form (0,1) and produce one output (0 or 1) after performing a boolean operation.

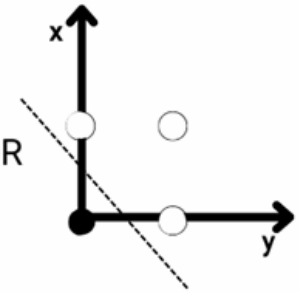
X1	X2	Y
F	F	0
F	T	0
T	F	0
T	T	1

AND



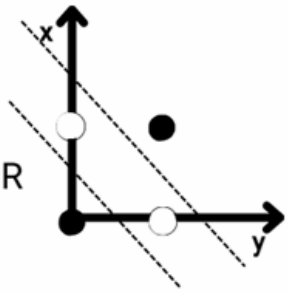
X1	X2	Y
F	F	0
F	T	1
T	F	1
T	T	1

OR



X1	X2	Y
F	F	0
F	T	1
T	F	1
T	T	0

XOR



A linear line can easily separate data points of OR and AND. So we can use a single-layer perceptron here.

Whereas, to separate data points of XOR, we need two linear lines or can add a new dimension and then separate them using a plane. Multi-layer Perceptron will work better in this case.

$$y = x_1\overline{x_2} + \overline{x_1}x_2$$

$$y = z_1 + z_2$$

where

$z_1 = x_1\overline{x_2}$ (function 1)

$z_2 = \overline{x_1}x_2$ (function 2)

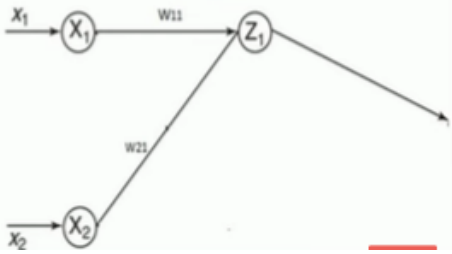
$y = z_1$ (OR) z_2 (function 3)

x_1	x_2	y
0	0	0
0	1	1
1	0	1
1	1	0

- First function $z_1 = x_1 \overline{x_2}$
- Assume the Initial weights are $w_{11} = w_{21} = 1$
- Threshold = 1 and learning rate = 1.5
- $(0,0) \rightarrow Z_{1in} = w_{ij} * x_i = 1 * 0 + 1 * 0 = 0 \ (out = 0)$
- $(0,1) \rightarrow Z_{1in} = w_{ij} * x_i = 1 * 0 + 1 * 1 = 1 \ (out = 1)$
  - $w_{ij} = w_{ij} + n(t - o)x_i$
  - $w_{11} = 1 + 1.5 * (0 - 1) * 0 = 0$
  - $w_{21} = 1 + 1.5 * (0 - 1) * 1 = -0.5$

X_1	X_2	Z_1
0	0	0
0	1	0
1	0	1
1	1	0

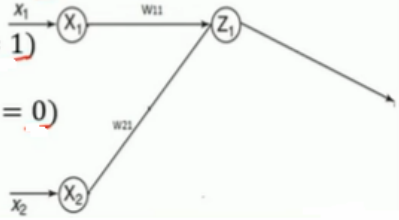
$$f(y_{in}) = \begin{cases} 1 & \text{if } y_{in} \geq \theta \\ 0 & \text{if } y_{in} < \theta \end{cases}$$



- First function $z_1 = x_1 \overline{x_2}$
- Assume the Initial weights are $w_{11} = 1 \ and \ w_{21} = -0.5$
- Threshold = 1 and learning rate = 1.5
- $(0,0) \rightarrow Z_{1in} = w_{ij} * x_i = 1 * 0 + (-0.5) * 0 = 0 \quad (out = 0)$
- $(0,1) \rightarrow Z_{1in} = w_{ij} * x_i = 1 * 0 + (-0.5) * 1 = -0.5 \ (out = 0)$
- $(1,0) \rightarrow Z_{1in} = w_{ij} * x_i = 1 * 1 + (-0.5) * 0 = 1 \quad (out = 1)$
- $(1,1) \rightarrow Z_{1in} = w_{ij} * x_i = 1 * 1 + (-0.5) * 1 = 0.5 \quad (out = 0)$

X_1	X_2	Z_1
0	0	0
0	1	0
1	0	1
1	1	0

$$f(y_{in}) = \begin{cases} 1 & \text{if } y_{in} \geq \theta \\ 0 & \text{if } y_{in} < \theta \end{cases}$$

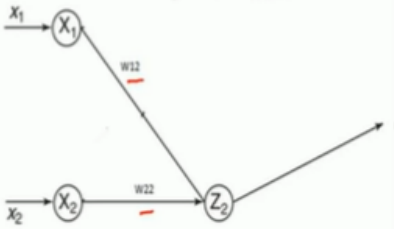


Now calculate Z2

- First function $z_2 = \overline{x_1} x_2$
- Assume the Initial weights are $w_{12} = w_{22} = 1$
- Threshold = 1 and learning rate = 1.5
- $(0,0) \rightarrow Z_{2in} = w_{ij} * x_i = 1 * 0 + 1 * 0 = 0 \quad (out = 0)$
- $(0,1) \rightarrow Z_{2in} = w_{ij} * x_i = 1 * 0 + 1 * 1 = 1 \quad (out = 1)$
- $(1,0) \rightarrow Z_{2in} = w_{ij} * x_i = 1 * 1 + 1 * 0 = 1 \quad (out = 1)$
  - $w_{ij} = w_{ij} + n(t - o)x_i$
  - $w_{12} = 1 + 1.5 * (0 - 1) * 1 = -0.5$
  - $w_{22} = 1 + 1.5 * (0 - 1) * 0 = 1$

X_1	X_2	Z_2
0	0	0
0	1	1
1	0	0
1	1	0

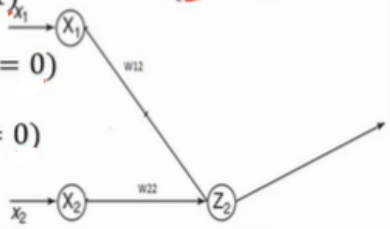
$$f(y_{in}) = \begin{cases} 1 & \text{if } y_{in} \geq \theta \\ 0 & \text{if } y_{in} < \theta \end{cases}$$



- First function $z_2 = \overline{x_1} x_2$
- Assume the Initial weights are $w_{12} = -0.5 \; and \; w_{22} = 1$
- Threshold = 1 and learning rate = 1.5
- $(0,0) \rightarrow Z_{2in} = w_{ij} * x_i = (-0.5) * 0 + 1 * 0 = 0 \quad (out = 0)$
- $(0,1) \rightarrow Z_{2in} = w_{ij} * x_i = (-0.5) * 0 + 1 * 1 = 1 \quad (out = 1)$
- $(1,0) \rightarrow Z_{2in} = w_{ij} * x_i = (-0.5) * 1 + 1 * 0 = -0.5 \quad (out = 0)$
- $(1,1) \rightarrow Z_{2in} = w_{ij} * x_i = (-0.5) * 1 + 1 * 1 = 0.5 \quad (out = 0)$

x_1	x_2	z_2
0	0	0
0	1	1
1	0	0
1	1	0

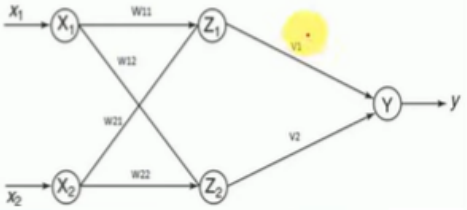
$$f(y_{in}) = \begin{cases} 1 & \text{if } y_{in} \geq \theta \\ 0 & \text{if } y_{in} < \theta \end{cases}$$



First function $y = z_1 \; (\text{OR}) \; z_2 \qquad y_{in} = z_1 v_1 + z_2 v_2$

x_1	x_2	z_1	z_2	y
0	0	0	0	0
0	1	0	1	1
1	0	1	0	1
1	1	0	0	0

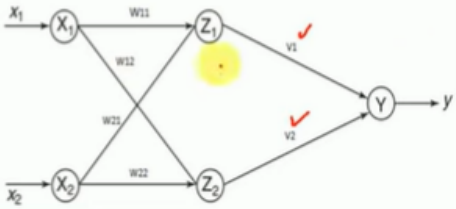
$$f(y_{in}) = \begin{cases} 1 & \text{if } y_{in} \geq \theta \\ 0 & \text{if } y_{in} < \theta \end{cases}$$



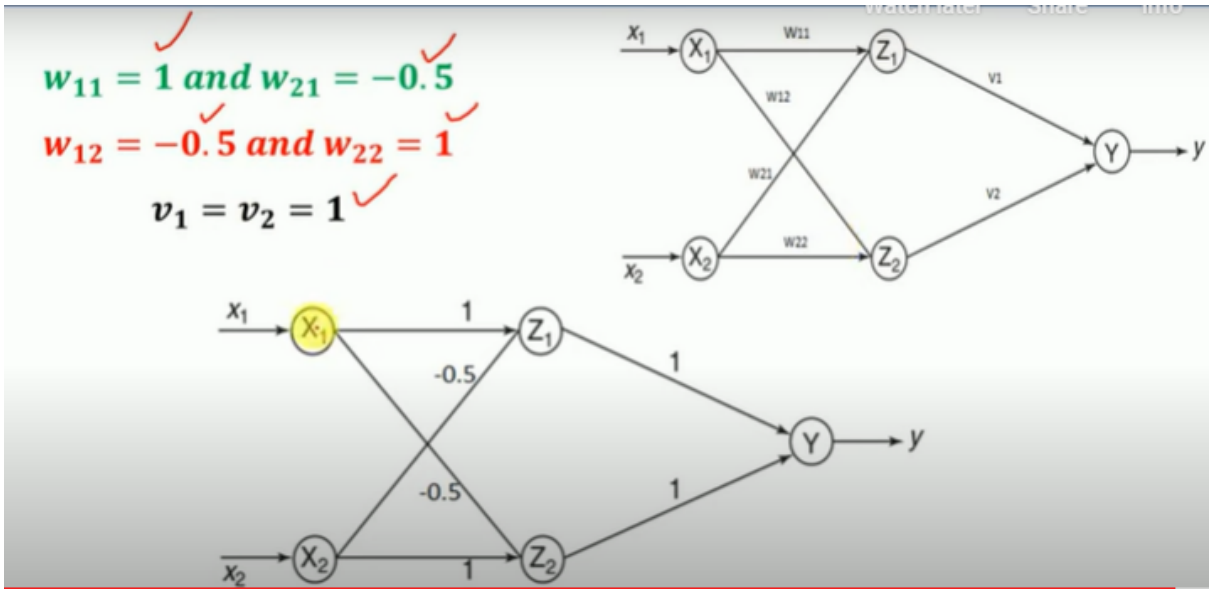
- First function $y = z_1 \; (\text{OR}) \; z_2 \qquad y_{in} = z_1 v_1 + z_2 v_2$
- Assume the Initial weights are $v_1 = v_2 = 1$
- Threshold = 1 and learning rate = 1.5
- $(0,0) \rightarrow y_{in} = v_i * x_i = 1 * 0 + 1 * 0 = 0 \quad (out = 0)$
- $(0,1) \rightarrow y_{in} = v_i * x_i = 1 * 0 + 1 * 1 = 1 \quad (out = 1)$
- $(1,0) \rightarrow y_{in} = v_i * x_i = 1 * 1 + 1 * 0 = 1 \quad (out = 1)$
- $(0,0) \rightarrow y_{in} = v_i * x_i = 1 * 0 + 1 * 0 = 0 \quad (out = 0)$

x_1	x_2	z_1	z_2	y
0	0	0	0	0
0	1	0	1	1
1	0	1	0	1
1	1	0	0	0

$$f(y_{in}) = \begin{cases} 1 & \text{if } y_{in} \geq \theta \\ 0 & \text{if } y_{in} < \theta \end{cases}$$



Final network



Multilayer Perceptron (MLP)

A multilayer perceptron is a type of feedforward neural network consisting of fully connected neurons with a nonlinear kind of activation function. It is widely used to distinguish data that is not linearly separable.

MLPs have been widely used in various fields, including image recognition, natural language processing, and speech recognition, among others. Their flexibility in architecture and ability to approximate any function under certain conditions make them a fundamental building block in deep learning and neural network research. Let's take a deeper dive into some of its key concepts.

Input layer

The input layer consists of nodes or neurons that receive the initial input data. Each neuron represents a feature or dimension of the input data. The number of neurons in the input layer is determined by the dimensionality of the input data.

Hidden layer

Between the input and output layers, there can be one or more layers of neurons. Each neuron in a hidden layer receives inputs from all neurons in the previous layer (either the input layer or another hidden layer) and produces an output that is passed to the next layer. The number of hidden layers and the number of neurons in each hidden layer are hyperparameters that need to be determined during the model design phase.

Output layer

This layer consists of neurons that produce the final output of the network. The number of neurons in the output layer depends on the nature of the task. In binary classification, there may be either one or two neurons depending on the activation function and representing the probability of belonging to one class; while in multi-class classification tasks, there can be multiple neurons in the output layer.

Weights

Neurons in adjacent layers are fully connected to each other. Each connection has an associated weight, which determines the strength of the connection. These weights are learned during the training process.

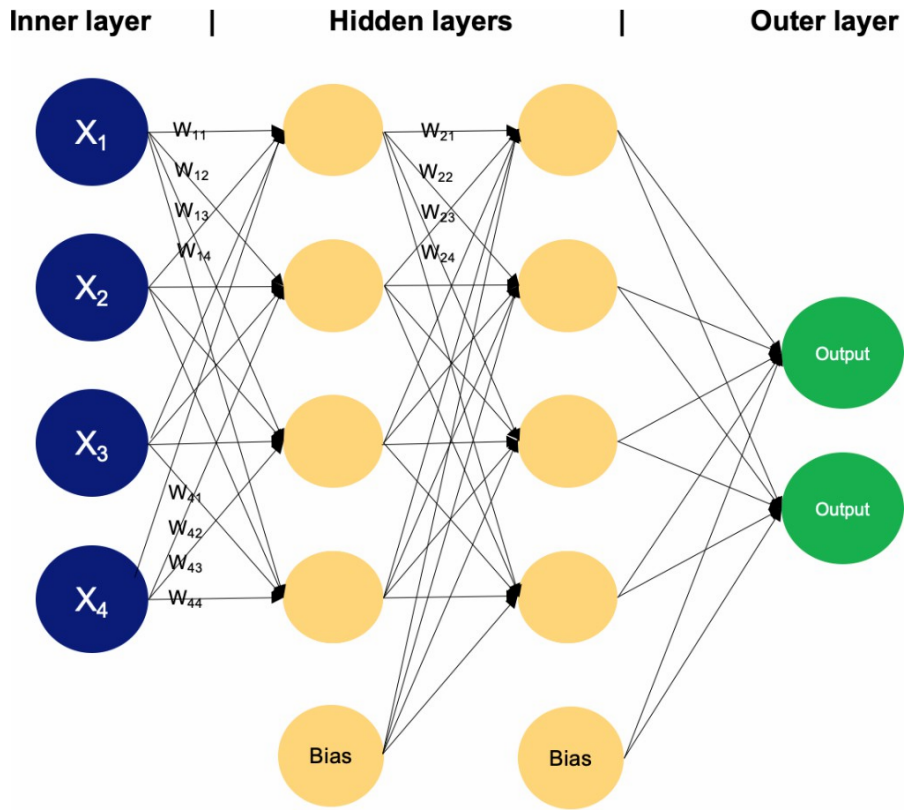
Bias neurons

In addition to the input and hidden neurons, each layer (except the input layer) usually includes a bias neuron that provides a constant input to the neurons in the next layer. Bias neurons have their own weight associated with each connection, which is also learned during training.

The bias neuron effectively shifts the activation function of the neurons in the subsequent layer, allowing the network to learn an offset or bias in the decision boundary. By adjusting the weights connected to the bias neuron, the MLP can learn to control the threshold for activation and better fit the training data.

Activation function

Typically, each neuron in the hidden layers and the output layer applies an activation function to its weighted sum of inputs. Common activation functions include sigmoid, tanh, ReLU (Rectified Linear Unit), and softmax. These functions introduce nonlinearity into the network, allowing it to learn complex patterns in the data.



Back propagation derivation

- To derive the equation for updating weights in back propagation algorithm, we use Stochastic gradient descent rule.
- Stochastic gradient descent involves iterating through the training examples one at a time, for each training example **d** descending the gradient of the error ***Ed*** with respect to this single example.
- In other words, for each training example **d** every weight ***wji*** is updated by adding to it Δw_{ji}.
- That is,

$$W_{ji} \leftarrow W_{ji} + \Delta W_{ji}$$

Where

$$\Delta w_{ji} = -\eta \frac{\partial E_d}{\partial w_{ji}}$$

$$w_{ji} \leftarrow w_{ji} + \Delta w_{ji}$$

Where

$$\Delta w_{ji} = -\eta \frac{\partial E_d}{\partial w_{ji}}$$

- where ***Ed*** is the error on training example **d**, that is half the squared difference between the target output and the actual output over all output units in the network,

$$E_d(\vec{w}) \equiv \frac{1}{2} \sum_{k \in outputs} (t_k - o_k)^2$$

- Here outputs is the set of output units in the network, $\mathbf{t_k}$ is the target value of unit k for training example **d**, and $\mathbf{o_k}$ is the output of unit k given training example **d**.

Notation Used:

$\boldsymbol{x_{ji}}$ = the ith input to unit j

$\boldsymbol{w_{ji}}$ = the weight associated with the ith input to unit j

$\boldsymbol{net_j} = \sum_i \boldsymbol{w_{ji} X_{ji}}$ (the weighted sum of inputs for unit j)

$\boldsymbol{o_j}$ = the output computed by unit j

$\boldsymbol{t_j}$ = the target output for unit j

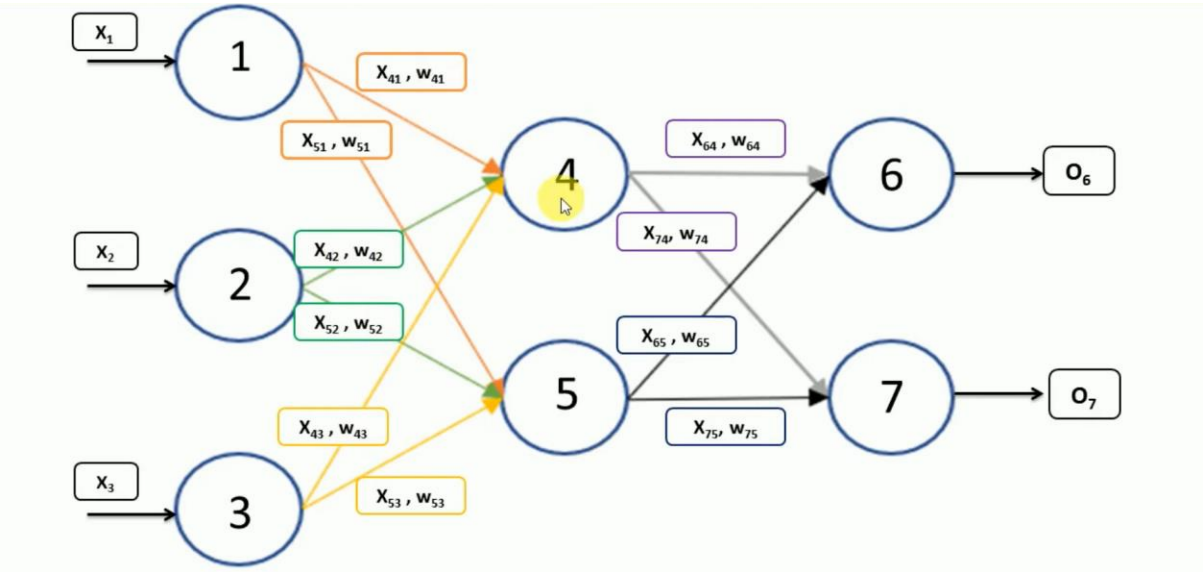
$\boldsymbol{\sigma}$ = the sigmoid function

outputs = the set of units in the final layer of the network

Downstream(j) = the set of units whose immediate inputs include the output of unit j

Downstream of 4 is 6,7 nodes

Downstream of 5 is 6,7 nodes



$$W_{ji} \leftarrow W_{ji} + \Delta W_{ji}$$

Where

$$\Delta w_{ji} = -\eta \frac{\partial E_d}{\partial w_{ji}}$$

- To begin, notice that weight ***wji*** can influence the rest of the network only through ***netj.*** Therefore, we can use the chain rule to write,

$$\frac{\partial E_d}{\partial w_{ji}} = \frac{\partial E_d}{\partial net_j} \frac{\partial net_j}{\partial w_{ji}}$$

$$= \frac{\partial E_d}{\partial net_j} x_{ji}$$

$$net_j = \sum_i w_{ji} X_{ji}$$

$$\frac{\partial net_j}{\partial w_{ji}} = x_{ji}$$

$$\Delta w_{ji} = -\eta \frac{\partial E_d}{\partial net_j} x_{ji}$$

- Our remaining task is to derive a convenient expression for $\frac{\partial E_d}{\partial net_j}$

To derive a convenient expression for $\frac{\partial E_d}{\partial net_j}$

We consider two cases in turn:

- Case 1, where unit j is an output unit for the network, and
- Case 2, where unit j is an internal unit of the network.

Case 1: Training Rule for Output Unit Weights

- Just as ***wji*** can influence the rest of the network only through net_j, net_j can influence the network only through ***oj.*** Therefore, we can invoke the chain rule again to write,

$$\frac{\partial E_d}{\partial net_j} = \frac{\partial E_d}{\partial o_j} \frac{\partial o_j}{\partial net_j}$$

Find out the differentiation of above two terms

$$\frac{\partial E_d}{\partial o_j} = \frac{\partial}{\partial o_j} \frac{1}{2} \sum_{k \in outputs} (t_k - o_k)^2$$

Keep the constant term outside to differentiate

$$\frac{1}{2}\sum_{k \in outputs}(t_k - o_k)^2$$

We are differentiating with respective to j here. Whenever the value k will become j then only this differentiation will give value otherwise it is zero

Here rather than considering the Summation term we consider k=j

$$\frac{\partial E_d}{\partial o_j} = \frac{\partial}{\partial o_j}\frac{1}{2}(t_j - o_j)^2$$

$$= \frac{1}{2}2(t_j - o_j)\frac{\partial(t_j - o_j)}{\partial o_j}$$

$$= -(t_j - o_j)$$

$$\frac{\partial o_j}{\partial (net_j)} = \frac{\partial \sigma(net_j)}{\partial (net_j)}$$

$$\frac{\partial \sigma(x)}{\partial (x)} = \sigma(x)\,(1 - \sigma(x))$$

$$= \sigma(net_j)\,(1 - \sigma(net_j))$$

$$= o_j\,(1 - o_j)$$

$$\frac{\partial E_d}{\partial net_j} = -(t_j - o_j)\; o_j(1 - o_j)$$

Case 1: Training Rule for Output Unit Weights

$$\Delta w_{ji} = -\eta \frac{\partial E_d}{\partial w_{ji}}$$

$$\Delta w_{ji} = -\eta \frac{\partial E_d}{\partial net_j} x_{ji}$$

$$\Delta w_{ji} = \eta\,(t_j - o_j)\; o_j(1 - o_j) x_{ji}$$

$$\Delta w_{ji} = \eta\, \delta_j x_{ji}$$

$$\frac{\partial E_d}{\partial net_j} = -(t_j - o_j)\; o_j(1 - o_j)$$

$$\delta_j = (t_j - o_j)\; o_j(1 - o_j)$$

Case 2: Training Rule for Hidden Unit Weights

$$\frac{\partial E_d}{\partial net_j} = \sum_{k \in Downstream(j)} \frac{\partial E_d}{\partial net_k} \frac{\partial net_k}{\partial net_j}$$

$$\frac{\partial E_d}{\partial net_j} = -(t_j - o_j)\, o_j(1 - o_j)$$

$$= \sum_{k \in Downstream(j)} -\delta_k \frac{\partial net_k}{\partial net_j}$$

$$= \sum_{k \in Downstream(j)} -\delta_k \frac{\partial net_k}{\partial o_j} \frac{\partial o_j}{\partial net_j}$$

$$= \sum_{k \in Downstream(j)} -\delta_k\, w_{kj} \frac{\partial o_j}{\partial net_j}$$

$$= \sum_{k \in Downstream(j)} -\delta_k\, w_{kj}\, o_j(1 - o_j)$$

$$\delta_j = (t_j - o_j)\, o_j(1 - o_j)$$

$$\frac{\partial net_k}{\partial o_j} = \frac{\partial x_{kj} w_{kj}}{\partial o_j} = \frac{\partial o_j w_{kj}}{\partial o_j}$$

$$\frac{\partial o_j}{\partial (net_j)} = \frac{\partial \sigma(net_j)}{\partial (net_j)}$$

$$= \sigma(net_j)\,(1 - \sigma(net_j))$$

$$= o_j\,(1 - o_j)$$

$$\Delta w_{ji} = -\eta \frac{\partial E_d}{\partial net_j} x_{ji}$$

$$\frac{\partial E_d}{\partial net_j} = \sum_{k \in Downstream(j)} -\delta_k\, w_{kj}\, o_j(1 - o_j)$$

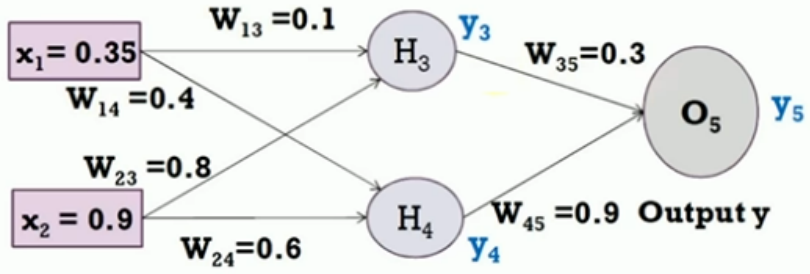
$$\Delta w_{ji} = \eta\, o_j(1 - o_j) \sum_{k \in Downstream(j)} \delta_k\, w_{kj}\, x_{ji}$$

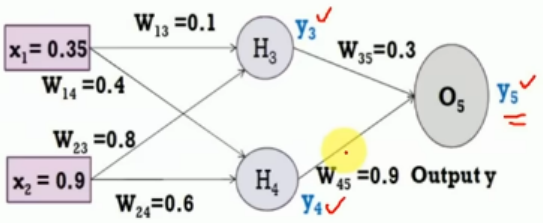
$$\Delta w_{ji} = \eta\, \delta_j\, x_{ji}$$

$$\delta_j = o_j(1 - o_j) \sum_{k \in Downstream(j)} \delta_k\, w_{kj}$$

Example:1

Assume that the neurons have a sigmoid activation function, perform a forward pass and a backward pass on the network. Assume that the actual output of y is 0.5 and learning rate is 1. Perform another forward pass.





- Forward Pass: Compute output for y3, y4 and y5.

$$a_j = \sum_j (w_{i,j} * x_i) \qquad yj = F(aj) = \frac{1}{1+e^{-a_j}}$$

$a_1 = (w_{13} * x_1) + (w_{23} * x_2)$
$= (0.1 * 0.35) + (0.8 * 0.9) = 0.755$
$y_3 = f(a_1) = 1/ (1 + e^{-0.755}) = 0.68$

$a_2 = (w_{14} * x_1) + (w_{24} * x_2)$
$= (0.4 * 0.35) + (0.6 * 0.9) = 0.68$
$y_4 = f(a_2) = 1/ (1 + e^{-0.68}) = 0.6637$

$a_3 = (w_{35} * y_3) + (w_{45} * y_4)$
$= (0.3 * 0.68) + (0.9 * 0.6637) = 0.801$
$y_5 = f(a_3) = 1/ (1 + e^{-0.801}) = \mathbf{0.69}$ (Network Output)

Error $= y_{target} - y_5 = -0.19$

0.5 - 0.69

BackPropagation

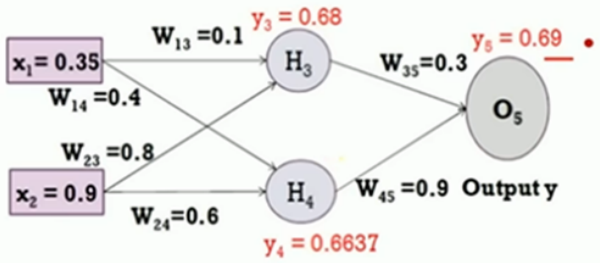
- Each weight changed by:

$$\Delta w_{ij} = \eta \delta_j o_i$$

$$\delta_j = o_j(1-o_j)(t_j - o_j) \qquad \text{if } j \text{ is an output unit}$$

$$\delta_j = o_j(1-o_j)\sum_k \delta_k w_{kj} \qquad \text{if } j \text{ is a hidden unit}$$

- where η is a constant called the learning rate
- tj is the correct output for unit j
- δj is the error measure for unit j



• Backward Pass: Compute δ3, δ4 and δ5.

For output unit:

$\delta_5 = y_5(1-y_5)(y_{target} - y_5)$

$= 0.69*(1-0.69)*(0.5-0.69) = -0.0406$

For hidden unit:

$\delta_3 = y_3(1-y_3)\, w_{35} * \delta_5$

$= 0.68*(1 - 0.68)*(0.3 * -0.0406) = -0.00265$

$\Delta w_{ij} = \eta \delta_j o_i$

$\delta_j = o_j(1-o_j)(t_j - o_j)$ if j is an output unit

$\delta_j = o_j(1-o_j)\sum_k \delta_k w_{kj}$ if j is a hidden unit

$\delta_4 = y_4(1-y_4)w_{45} * \delta_5$

$= 0.6637*(1 - 0.6637)* (0.9 * -0.0406) = -0.0082$

Compute new weights

$$\Delta w_{ij} = \eta \delta_j o_i$$

$\Delta w_{45} = \eta\, \delta_5\, y_4 = 1 * -0.0406 * 0.6637 = -0.0269$

w_{45} (new) = $\Delta w_{45} + w_{45}$(old) = -0.0269 + (0.9) = 0.8731

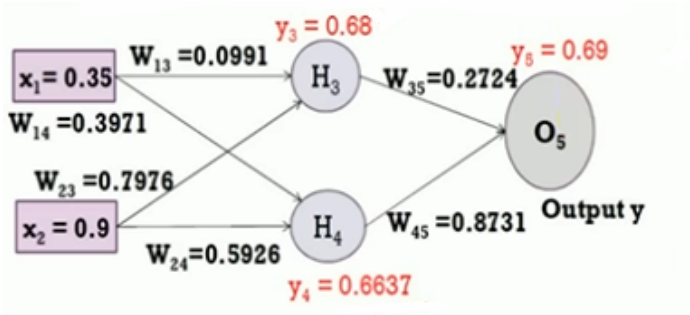
$\Delta w_{14} = \eta\, \delta_4\, x_1 = 1 * -0.0082 * 0.35 = -0.00287$

w_{14} (new) = $\Delta w_{14} + w_{14}$(old) = -0.00287+ 0.4 = 0.3971

- Similarly, update all other weights

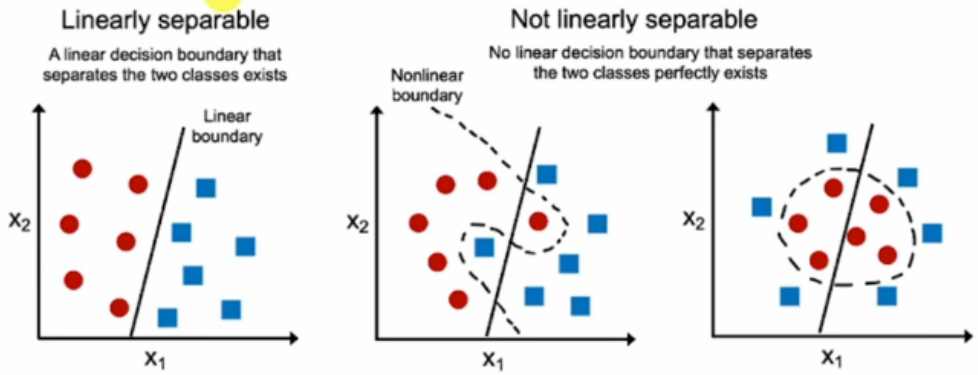
i	j	w_{ij}	δ_j	x_i	η	Updated w_{ij}
1	3	0.1	-0.00265	0.35	1	0.0991
2	3	0.8	-0.00265	0.9	1	0.7976
1	4	0.4	-0.0082	0.35	1	0.3971
2	4	0.6	-0.0082	0.9	1	0.5926
3	5	0.3	-0.0406	0.68	1	0.2724
4	5	0.9	-0.0406	0.6637	1	0.8731

Network after changing the weights



Radial Basis Function (RBF)

- Data can be either linearly Separable or Non-Linearly Separable
- Single layer perceptron can be used for classifying linearly Separable data.
- Multi layer perceptron is required for classifying Non-linearly Separable data.

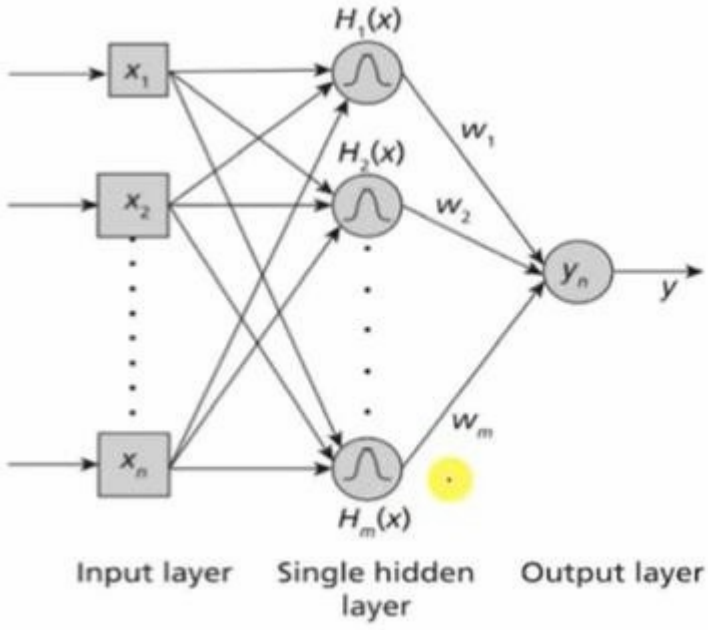


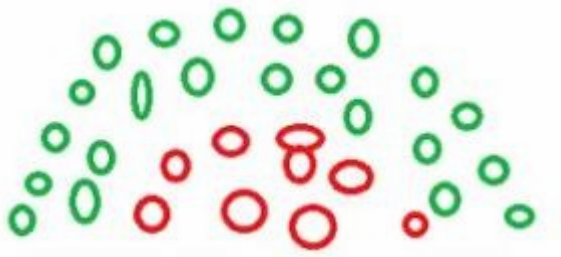
Radial Basis Functions (RBFs) are a special category of feed-forward neural networks comprising three layers:

1. Input Layer: Receives input data and passes it to the hidden layer.
2. **Hidden Layer**: The core computational layer where RBF neurons process the data.
3. **Output Layer**: Produces the network's predictions, suitable for classification or regression tasks.
   RBF contains strictly one hidden layer.

1. **Input Vector**: The network receives an n-dimensional input vector that needs classification or regression.

2. **RBF Neurons**: Each neuron in the hidden layer represents a prototype vector from the training set. The network computes the Euclidean distance between the input vector and each neuron's center.
3. **Activation Function**: The Euclidean distance is transformed using a Radial Basis Function (typically a Gaussian function) to compute the neuron's activation value. This value decreases exponentially as the distance increases.
4. **Output Nodes**: Each output node calculates a score based on a weighted sum of the activation values from all RBF neurons. For classification, the category with the highest score is chosen.





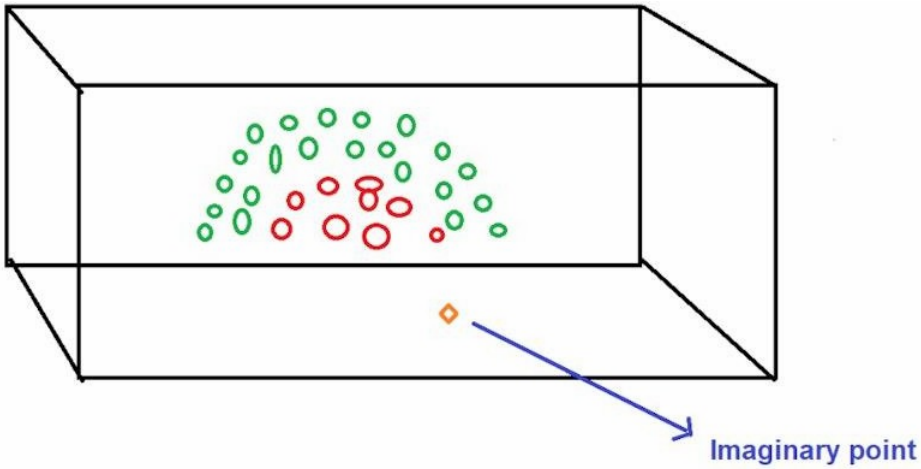
It is clear that we cannot use a linear hyperplane such that it can group the data points according to their classes.

RBF will help us in these kinds of scenarios.

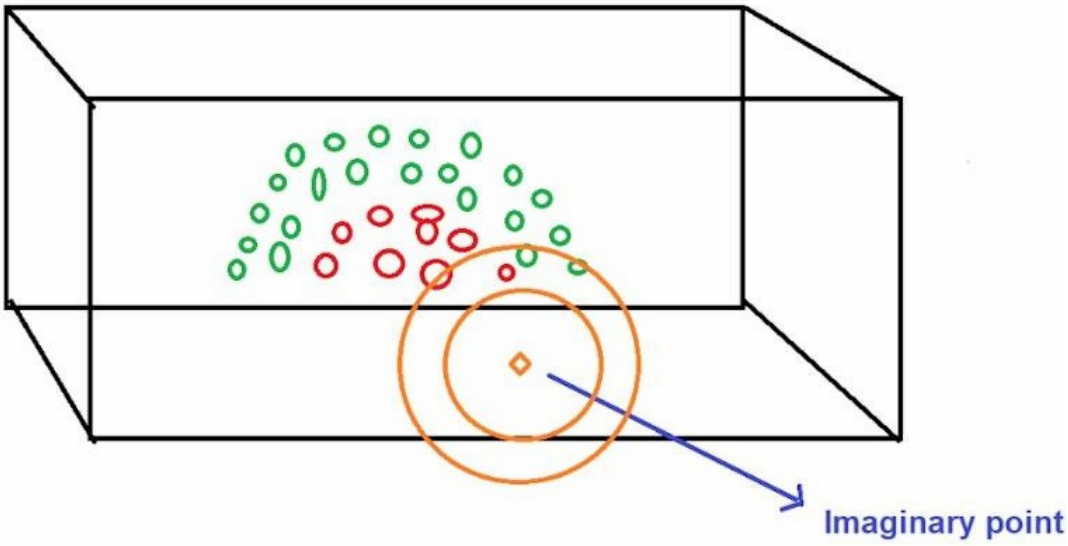
Some researchers will usually project these data points in much higher dimensions so that the distance between the data points will be increasing so that they can apply some function (RBF or any other

function) to build a hyperplane. But it is not necessary to build high dimensions since it is always the decision of the statistician/researcher who understands the patterns in the data.

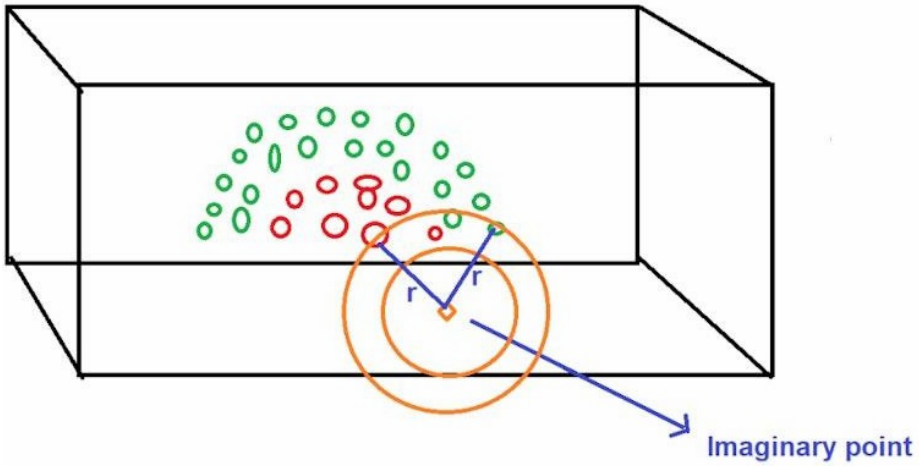
Next, we have to mark an imaginary point in the space like this wherever we need.



After that, we need to draw some concentric circles based on this imaginary point.

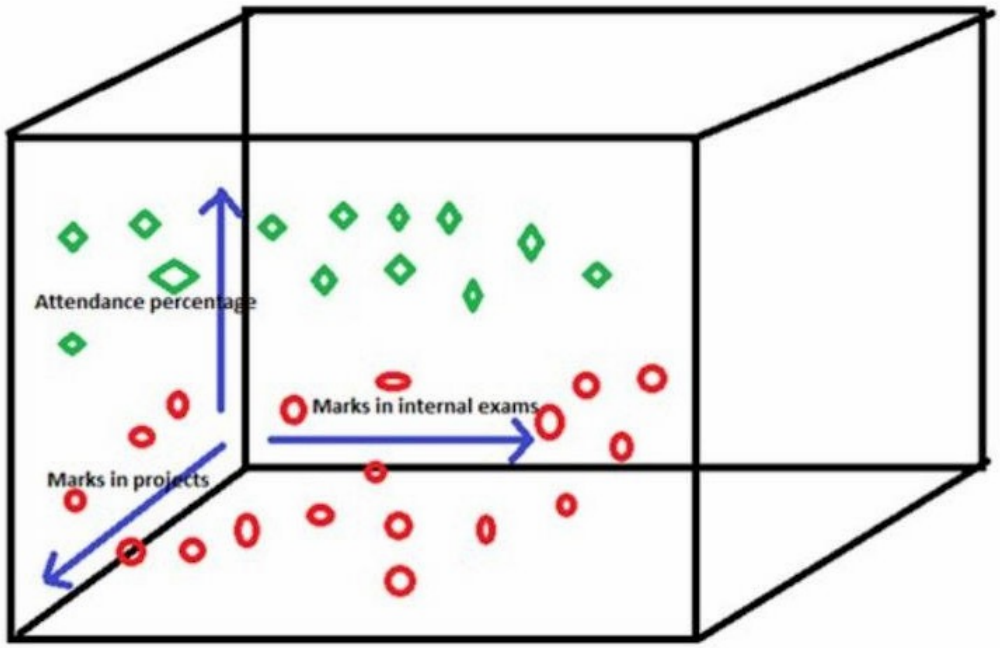


The distance between the centre and any data point positioned in the boundary of the circle is called the radius.

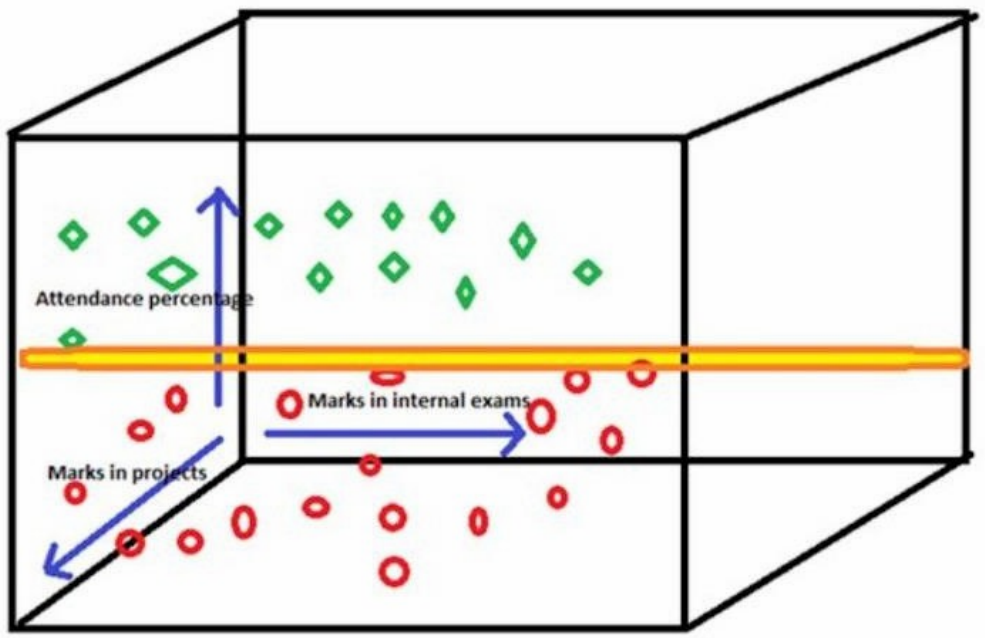


After calculating the radius, we need to pass this value inside a mathematical function (RBF) that will return a real value. The returned value will be the transformed magnitude of a particular data point used for further proceedings.

After the expansion and compression, the data points would have been transformed like this-



Now, we can easily construct a linear hyperplane that can classify the data points like this-



Types of Radial Basis Functions

There are multiple types of radial basis functions. Each of them will transform the input value in a different way. Some of them are-

- The **Gaussian RBF** which monotonically decreases with distance from the center.

$$H(x) = e^{\frac{-(x-c)^2}{r^2}}$$

- where, c is the center and r is the radius.
- A **Multiquadric RBF** which monotonically increases with distance from the center.

$$H(x) = \sqrt{\frac{r^2 + (x-c)^2}{r}}$$

Input: Input vector $(x_1, x_2, ..., x_n)$

Output: Y_n

Assign random weights for every connection from the Hidden layer to the Output layer in the network in the range [−1, +1].

Forward Phase:

Step 1: Calculate Input and Output in the Input Layer:

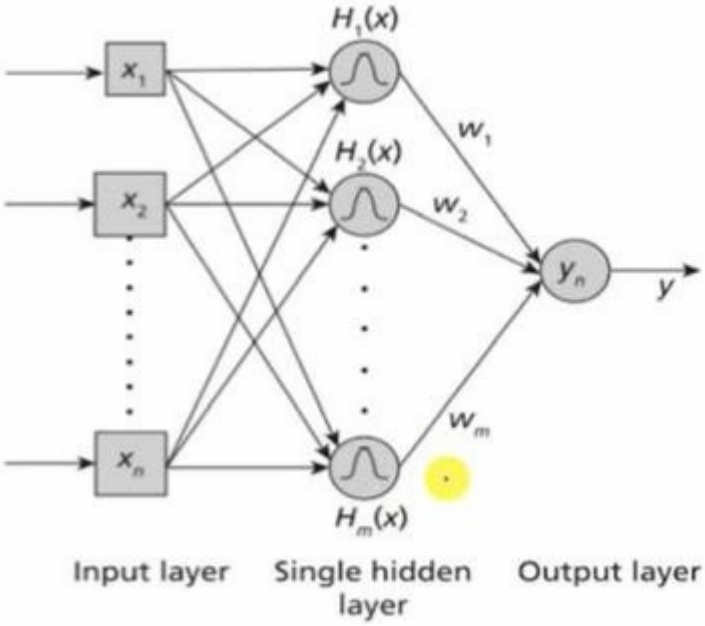
(Input layer is a direct transfer function, where the output of the node equals the input).

Input at Node i 'I$_i$' in the Input Layer is

$$I_i = x_i$$

where,

x_i is the input received at Node *i*. *Output at Node i 'O$_i$' in the Input Layer is* $O_i=I_i$



Step 2: For each node *j* in the Hidden Layer, find the centre/receptor *c* and the variance *r*.

Define hidden layer neurons with Gaussian RBF whose output is:

$$H_j(x) = e^{\frac{-(x-c_j)^2}{r^2}}$$

where, x is the input, c_j is the centre and r is the radius.

Compute $(x - c_j)^2$ applying Euclidean distance measure between x and c_j.

Step 3: For each node *k* in the Output Layer, compute linear weighted sum of the output of each neuron *k* from the hidden layer neurons *j*.

$$F_k(x) = \sum_{j=1}^{m} w_{jk} H_j(x)$$

where,

w_{jk} is the weight in the link from the Hidden Layer neuron *j* to the Output Layer neuron *k*.

$H_j(x)$ is the output of a Hidden Layer neuron *j* for an input vector x.

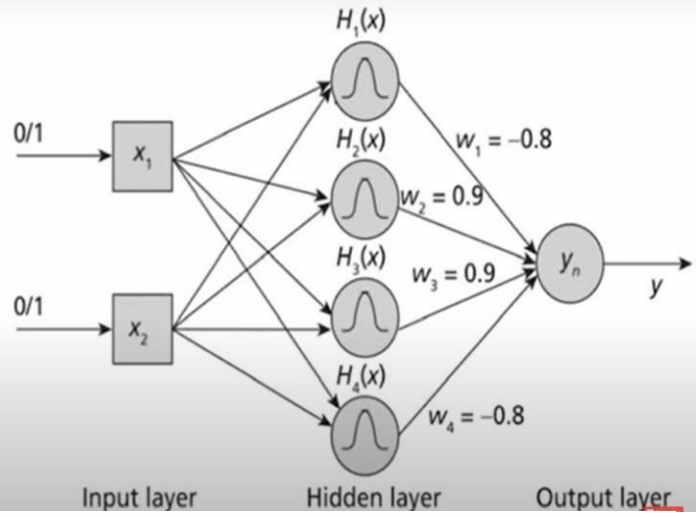
Backward Phase:

Step 1: Train the Hidden layer using Back propagation.

Step 2: Update the weights between the Hidden layer and Output layer.

Example

Consider the XOR Boolean function that has 4 patterns (0, 0) (0, 1) (1, 0) and (1, 1) in a 2-D input space.



- Radius $r = 2$
- Construct a RBFNN as shown in Figure that classifies the input pattern:
- (0, 0) → 0
- (0, 1) → 1
- (1, 0) → 1
- (1, 1) → 0

Define 4 hidden layer neurons with Gaussian RBF:

$$H_1(x) = e^{\frac{-(x-c1)^2}{r^2}} ; c1 = (0, 0)$$

$$H_2(x) = e^{\frac{-(x-c2)^2}{r^2}} ; c2 = (0, 1)$$

$$H_3(x) = e^{\frac{-(x-c3)^2}{r^2}} ; c3 = (1, 0)$$

$$H_4(x) = e^{\frac{-(x-c4)^2}{r^2}} ; c4 = (1, 1)$$

Where x is the input, c1 is the centre, r is the radius

For the first neuron in hidden layer c1(0,0) is considered as the centre , In the 2nd neuron c2 (0,1)and 3rd neuron c3 (1,0)and 4th neuron c4(1,1).

To calculate numerator value we use Euclidian distance formula

$$d^2 = (x_2 - x_1)^2 + (y_2 - y_1)^2$$

For input pattern (0, 0)

Distance squared of x from $c1 = (0, 0)$

$= (0 - 0)^2 + (0 - 0)^2 = 0$

$$H_1(x) = e^{\frac{-(x-c1)^2}{r^2}} = e^{-\frac{0}{2}} = 1.0$$

Distance squared of x from $c2 = (0, 1)$

$= (0 - 0)^2 + (0 - 1)^2 = 1$

$$H_2(x) = e^{\frac{-(x-c2)^2}{r^2}} = e^{-\frac{1}{2}} = 0.6$$

Distance squared of x from $c3 = (1, 0)$

$= (0 - 1)^2 + (0 - 0)^2 = 1$

$$H_3(x) = e^{\frac{-(x-c3)^2}{r^2}} = e^{-\frac{1}{2}} = 0.6$$

Distance squared of x from $c4 = (1, 1)$

$= (0 - 1)^2 + (0 - 1)^2 = 2$

$$H_4(x) = e^{\frac{-(x-c4)^2}{r^2}} = e^{-\frac{2}{2}} = 0.4$$

$$\sum_{j=1}^{m} w_j H_j(x) = -0.8 \times 1.0 + 0.9 \times 0.6 + 0.9 \times 0.6 +$$

$$-0.8 \times 0.4 = -0.04$$

For input pattern (0, 1)

Distance squared of x from $c1 = (0, 0)$

$= (0 - 0)^2 + (1 - 0)^2 = 1$

$$H_1(x) = e^{\frac{-(x-c1)^2}{r^2}} = e^{-\frac{1}{2}} = 0.6$$

Distance squared of x from $c2 = (0, 1)$

$= (0 - 0)^2 + (1 - 1)^2 = 0$

$$H_2(x) = e^{\frac{-(x-c2)^2}{r^2}} = e^{-\frac{1}{2}} = 1.0$$

Distance squared of x from $c3 = (1, 0)$

$= (0 - 1)^2 + (1 - 0)^2 = 2$

$$H_3(x) = e^{\frac{-(x-c3)^2}{r^2}} = e^{-\frac{2}{2}} = 0.4$$

Distance squared of x from $c4 = (1, 1)$

$= (0 - 1)^2 + (1 - 1)^2 = 1$

$$H_4(x) = e^{\frac{-(x-c4)^2}{r^2}} = e^{-\frac{1}{2}} = 0.6$$

$\sum_{j=1}^{m} w_j H_j(x) = -0.8 \times 0.6 + 0.9 \times 1.0 + 0.9 \times 0.4 +$

$-0.8 \times 0.6 = 0.3$

For input pattern (1, 0)

Distance squared of x from $c1 = (0, 0)$

$= (1 - 0)^2 + (0 - 0)^2 = 1$

$$H_1(x) = e^{\frac{-(x-c1)^2}{r^2}} = e^{-\frac{1}{2}} = 0.6$$

Distance squared of x from $c2 = (0, 1)$

$= (1 - 0)^2 + (0 - 1)^2 = 2$

$$H_2(x) = e^{\frac{-(x-c2)^2}{r^2}} = e^{-\frac{2}{2}} = 0.4$$

Distance squared of x from $c3 = (1, 0)$

$= (1 - 1)^2 + (0 - 0)^2 = 0$

$$H_3(x) = e^{\frac{-(x-c3)^2}{r^2}} = e^{-\frac{1}{2}} = 1.0$$

Distance squared of x from $c4 = (1, 1)$

$= (1 - 1)^2 + (0 - 1)^2 = 1$

$$H_4(x) = e^{\frac{-(x-c4)^2}{r^2}} = e^{-\frac{1}{2}} = 0.6$$

$\sum_{j=1}^{m} w_j H_j(x) = -0.8 \times 0.6 + 0.9 \times 0.4 + 0.9 \times 1.0 +$

$-0.8 \times 0.6 = 0.3$

For input pattern (1, 1)

Distance squared of x from $c1 = (0, 0)$

$= (1 - 0)^2 + (1 - 0)^2 = 2$

$$H_1(x) = e^{\frac{-(x-c1)^2}{r^2}} = e^{-\frac{2}{2}} = 0.4$$

Distance squared of x from $c2 = (0, 1)$

$= (1 - 0)^2 + (1 - 1)^2 = 1$

$$H_2(x) = e^{\frac{-(x-c2)^2}{r^2}} = e^{-\frac{1}{2}} = 0.6$$

Distance squared of x from $c3 = (1, 0)$

$= (1 - 1)^2 + (1 - 0)^2 = 1$

$$H_3(x) = e^{\frac{-(x-c3)^2}{r^2}} = e^{-\frac{1}{2}} = 0.6$$

Distance squared of x from $c4 = (1, 1)$

$= (1 - 1)^2 + (1 - 1)^2 = 0$

$$H_4(x) = e^{\frac{-(x-c4)^2}{r^2}} = e^{-\frac{0}{2}} = 1.0$$

$$\sum_{j=1}^{m} w_j H_j(x) = -0.8 \times 0.4 + 0.9 \times 0.6 + 0.9 \times 0.6 + -0.8 \times 1.0 = -0.04$$

Forward Phase Calculation

Input		$H_1(x)$	$H_2(x)$	$H_3(x)$	$H_4(x)$	$\sum_{j=1}^{m} w_j H_j(x)$	Output
0	0	1.0	0.6	0.6	0.4	–0.04	0
0	1	0.6	1.0	0.4	0.6	0.3	1
1	0	0.6	0.4	1.0	0.6	0.3	1
1	1	0.4	0.6	0.6	1.0	–0.04	0
		$w_1 = -0.8$	$w_2 = 0.9$	$w_3 = 0.9$	$w_4 = -0.8$		

For (0,0) and (1,1) input the output is -0.04. This is less than 0 so out put is 0

For (0,1) and (1,0) input the output is 0.3. This is greater than 0 so out put is 1

The expected value we got here. So no need to do backpropagation.

Face Recognition

Face recognition is the process of identifying a person from an image or video feed and face detection is the process of detecting a face in an image or video feed. In the case of Face recognition, someone's face is recognized and differentiated based on their facial features. It involves more advanced processing techniques to identify a person's identity based on feature point extraction, and comparison algorithms. and can be used for applications such as automated attendance systems or security checks.

Image Processing and Machine learning

Image processing by computers involves the process of Computer Vision. It deals with a high-level understanding of digital images or videos. The requirement is to automate tasks that the human visual systems can do. So, a computer should be able to recognize objects such as the face of a human being or a lamppost, or even a statue.

Image reading:

The computer reads any image in a range of values between **0 and 255**. For any color image, there are 3 primary colors – Red, green, and blue. A matrix is formed for every primary color and later these matrices combine to provide a Pixel value for the individual R, G, and B colors. Each element of the matrices provide data about the **intensity of the brightness of the pixel.**

OpenCV is a Python library that is designed to solve **computer vision problems.**

Machine learning

Every Machine Learning algorithm takes a dataset as input and learns from the data it basically means to learn the algorithm from the provided input and output as data. It identifies the patterns in the data and provides the desired algorithm. For instance, to identify whose face is present in a given image, multiple things can be looked at as a pattern:

- Height/width of the face.
- Height and width may not be reliable since the image could be rescaled to a smaller face or grid. However, even after rescaling, what remains unchanged are the ratios – the ratio of the height of the face to the width of the face won't change.
- Color of the face.

- Width of other parts of the face like lips, nose, etc.

As a simple example, we can map a “face” into a feature vector which can comprise various features like:

- Height of face (cm)
- Width of the face (cm)
- Average color of face (R, G, B)
- Width of lips (cm)
- Height of nose (cm)

Essentially, given an image, we can convert them into a feature vector like:

Height of face (cm) Width of the face (cm) Average color of face (RGB) Width of lips (cm) Height of nose (cm)

23.1 15.8 (255, 224, 189) 5.2 4.4

So, the image is now a vector that could be represented as (23.1, 15.8, 255, 224, 189, 5.2, 4.4). There could be countless other features that could be derived from the image,, for instance, hair color, facial hair, spectacles, etc.

Machine Learning does two major functions in face recognition technology. These are given below:

1. Deriving the feature vector: it is difficult to manually list down all of the features because there are just so many. A Machine Learning algorithm can intelligently label out many of such features. For instance, a complex feature could be the ratio of the height of the nose and the width of the forehead.
2. Matching algorithms: Once the feature vectors have been obtained, a Machine Learning algorithm needs to match a new image with the set of feature vectors present in the corpus.
3. Face Recognition Operations

Face Recognition Operations

The technology system may vary when it comes to facial recognition. Different software applies different methods and means to achieve face recognition. The stepwise method is as follows:

- ***Face Detection:*** To begin with, the camera will detect and recognize a face. The face can be best detected when the person is looking directly at the camera as it makes it easy for facial recognition. With the advancements in technology, this is improved where the face can be detected with slight variation in their posture of face facing the camera.
- ***Face Analysis:*** Then the photo of the face is captured and analyzed. Most facial recognition relies on 2D images

- ***Image to Data Conversion:*** Now it is converted to a mathematical formula and these facial features become numbers

- ***Match Finding:*** Then the code is compared against a database of other face prints. This database has photos with identification that can be compared. The technology then identifies a match for your exact features in the provided database. It returns with the match and attached information such as name and address or it depends on the information saved in the database of an individual.

Face recognition software: Deepvision A, Sense time,Amazon Rekognition, Face forst, truface, cognitec, etc

Applications:

Hospitals, Airline, Universities etc